



2025/12/17

姓名：陳篁瑋

學號：01257024

班級：資工 3A

E-mail：

henry246813579@gmail.com

● 實驗說明：

1. 請透過當前課堂所學的所有組合語言撰寫一個程式計算兩個矩陣 A 和 B 的乘積，並將結果矩陣 C 儲存於記憶體位址 0x12 起的位置。矩陣 A 和 B 的元素皆為有號數 1 Byte，而矩陣 C 的元素為有號數 2 Bytes。程式需依據指定的矩陣儲存格式進行運算，如圖一所示。程式碼須加註解。

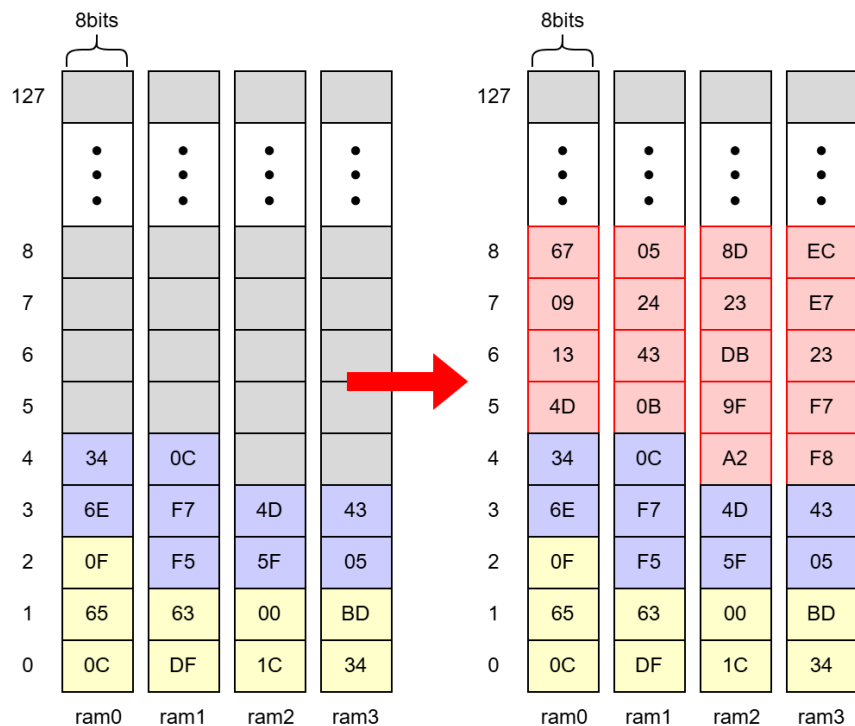
矩陣 A

矩陣 B

矩陣 C

$$\begin{bmatrix} 0C & DF & 1C \\ 34 & 65 & 63 \\ 00 & BD & 0F \end{bmatrix} \times \begin{bmatrix} F5 & 5F & 05 \\ 6E & F7 & 4D \\ 43 & 34 & 0C \end{bmatrix} = \begin{bmatrix} F8A2 & 0B4D & F79F \\ 4313 & 23DB & 2409 \\ E723 & 0567 & EC8D \end{bmatrix}$$

2. 所有暫存器皆可使用，但請將結果依序(0xF8A2→0x0B4D →... →0x0567 →0xEC8D)載入到 x31 暫存器中。
3. 程式撰寫完成後透過 Tool Chain 將其轉換成 Program_ROM.sv 檔，並使用 ModelSim 軟體觀察波形是否執行正確。
4. 請將所有用到的暫存器都加入至波形中以方便觀察。



圖一

```

init:
    li x2, 0x341CDF0C
    sw x2, 0(x0)
    li x2, 0xBD006365
    sw x2, 4(x0)
    li x2, 0xF
    sb x2, 8(x0)
    li x3, 0x9
    li x2, 0x6E055FF5
    sw x2, 0(x3)
    li x2, 0x34434DF7
    sw x2, 4(x3)
    li x2, 0xC
    sb x2, 8(x3)

#-----開始撰寫您的組合語言-----

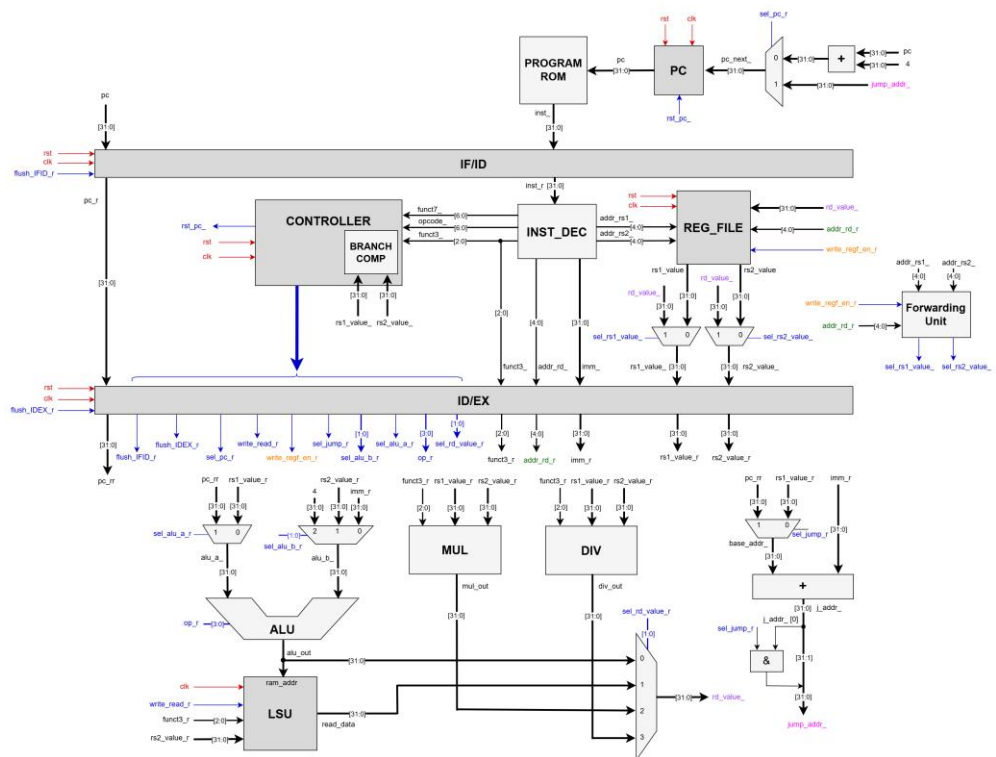
#-----您的組合語言-----

    li x3, 0x12
    li x4, 9
done:
    lh x31, 0(x3)
    addi x3, x3, 2
    addi x4, x4, -1
    bne x4, x0, done

```

圖二

● 系統硬體架構方塊圖（接線圖）：



● 系統架構程式碼、測試資料程式碼與程式碼說明

截圖請善用 win+shift+S

RISC-V

```

1  module risc_v(
2      input logic clk,
3      input logic rst,
4      //
5      output logic [31:0] regs_31
6      //
7  );
8      logic [31:0] rs1_value_;
9      logic [31:0] rs2_value_;
10     logic [31:0] rs1_value;
11     logic [31:0] rs2_value;
12     logic [31:0] pc;
13     logic [31:0] pc_next_;
14     logic rst_pc_;
15     logic sel_pc_r;
16     logic flush_IDEX_r;
17     logic flush_IFID_r;
18     //加法器
19     //assign pc_next_ = pc+4;
20     logic [31:0] pc_r, pc_rr;
21     //program counter
22     always_ff@(posedge clk)
23     begin
24         if(rst|rst_pc_)
25             pc <= 32'b0;
26         else
27             pc <= pc_next_;
28     end
29
30     logic [31:0] inst_;
31     Program_Rom Program_Rom_1(
32         .Rom_addr    (pc),
33         .Rom_data    (inst_)
34     );
35
36     // IF/ID
37     logic [31:0] inst_r;
38     logic flush_IFID_;

```

```

39     always_ff@(posedge clk)
40     begin
41         if(rst)begin
42             inst_r <= 32'h13;
43             pc_r <= 0;
44         end
45         else if(flush_IFID_r)begin
46             inst_r <= 32'h13;
47             pc_r <= 0;
48         end
49         else
50         begin
51             inst_r <= inst_;
52             pc_r <= pc;
53         end
54     end
55     define F_BEQ      3'b000
56     define F_BNE      3'b001
57     define F_BLT      3'b100
58     define F_BGE      3'b101
59     define F_BLTU     3'b110
60     define F_BGEU     3'b111
61     define F_LB       3'b000
62     define F_LH       3'b001
63     define F_LW       3'b010
64     define F_LBU      3'b100
65     define F_LHU      3'b101
66     define F_SB       3'b000
67     define F_SH       3'b001
68     define F_SW       3'b010
69     define F_MUL      3'b000
70     define F_MULH     3'b001
71     define F_MULHSU   3'b010
72     define F_MULHU    3'b011
73     define F_DIV      3'b100
74     define F_DIVU     3'b101
75     define F_REM      3'b110
76     define F_REMU     3'b111

```

```

77 //inst_decoder
78
79 `define opcode_I      7'b0010011
80 `define opcode_R      7'b0110011
81 `define opcode_B      7'b1100011
82 `define opcode_JAL    7'b1101111
83 `define opcode_JALR   7'b1100111
84 `define opcode_LUI     7'b0110111
85 `define opcode_AUIPC   7'b0110111
86 `define opcode_L       7'b0010011
87 `define opcode_S       7'b0100011
88 `define F7_M           7'b0000001
89 `define opcode_M       7'b0110011
90
91 logic [6:0] opcode_, funct7_;
92 logic [4:0] addr_rd_;
93 logic [2:0] funct3_;
94 logic [4:0] addr_rs1_, addr_rs2_;
95 logic [31:0] imm_;
96
97 assign opcode_ = inst_r[6:0];
98 assign addr_rd_ = inst_r[11:7];
99 assign funct3_ = inst_r[14:12];
100 assign addr_rs1_ = inst_r[19:15];
101 assign addr_rs2_ = inst_r[24:20];
102 assign funct7_ = inst_r[31:25];
103
104 logic[31:0] IMM_I;
105 logic[31:0] IMM_B;
106 logic[31:0] IMM_JAL;
107 logic[31:0] IMM_LUI_AUIPC;
108 logic[31:0] IMM_S;
109 assign IMM_I = {{20{inst_r[31]}}, inst_r[31:20]};
110 assign IMM_B = {{20{inst_r[31]}}, inst_r[7], inst_r[30:25], inst_r[11:8], 1'b0};
111 assign IMM_JAL = {{12{inst_r[31]}}, inst_r[19:12], inst_r[20], inst_r[30:21], 1'b0};
112 assign IMM_LUI_AUIPC = {inst_r[31:12], 12'b0};
113 assign IMM_S = {{20{inst_r[31]}}, inst_r[31:25], inst_r[11:7]};
114

```

```

115
116 always_comb begin
117     unique case (opcode_)
118         opcode_I : imm_ = IMM_I;
119         opcode_B : imm_ = IMM_B;
120         opcode_JAL : imm_ = IMM_JAL;
121         opcode_JALR : imm_ = IMM_I;
122         opcode_LUI : imm_ = IMM_LUI_AUIPC;
123         opcode_AUIPC : imm_ = IMM_LUI_AUIPC;
124         opcode_L : imm_ = IMM_I;
125         opcode_S : imm_ = IMM_S;
126     endcase
127 end
128
129 // ALU operation
130
131 `define ALUOP_ADD      4'h0
132 `define ALUOP_SUB      4'h1
133 `define ALUOP_AND      4'h2
134 `define ALUOP_OR       4'h3
135 `define ALUOP_XOR      4'h4
136 `define ALUOP_A        4'h5
137 `define ALUOP_A_ADD_4  4'h6
138 `define ALUOP_LTU      4'h7
139 `define ALUOP_LT       4'h8
140 `define ALUOP_SLL      4'h9
141 `define ALUOP_SRL      4'hA
142 `define ALUOP_SRA      4'hB
143 `define ALUOP_B        4'hC
144
145 //controller
146
147 logic BEQ_FLAG;
148 logic BNE_FLAG;
149 logic BLT_FLAG;
150 logic BGE_FLAG;
151 logic BLTU_FLAG;
152 logic BGEU_FLAG;

```

```

153
154 assign BEQ_FLAG = (rs1_value_ == rs2_value_);
155 assign BNE_FLAG = (rs1_value_ != rs2_value_);
156 assign BLT_FLAG = ($signed(rs1_value_) < $signed(rs2_value_));
157 assign BGE_FLAG = ($signed(rs1_value_) >= $signed(rs2_value_));
158 assign BLTU_FLAG = (rs1_value_ < rs2_value_);
159 assign BGEU_FLAG = (rs1_value_ >= rs2_value_);
160
161 logic flush_IDEX_;
162 logic write_regf_en_;
163 logic sel_pc_;
164 logic sel_alu_a_;
165 logic [1:0] sel_alu_b_;
166 logic sel_jump_;
167 logic [3:0] op_;
168 logic write_ram_;
169 logic [1:0] sel_rd_value_;
170 typedef enum {S0, S1, S2, S3}FSM_STATE;
171 FSM_STATE ps, ns;
172
173 always_ff@(posedge clk)
174 begin
175     if(rst)
176         ps <= #1 S0;
177     else
178         ps <= #1 ns;
179 end
180
181 always_comb
182 begin
183     write_ram_ = 0;
184     sel_rd_value_ = 0;
185     write_regf_en_ = 0;
186     sel_jump_ = 1;
187     rst_pc_ = 0;
188     flush_IFID_ = 0;
189     flush_IDEX_ = 0;
190     op_ = 0;

```

```

191     sel_alu_b_ = 0;
192     sel_alu_a_ = 0;
193     sel_pc_ = 0;
194     ns = ps;
195     unique case(ps)
196     S0:
197     begin
198         flush_IFID_ = 1;
199         flush_IDEX_ = 1;
200         rst_pc_ = 1;
201         ns = S1;
202     end
203     S1:
204     begin
205         flush_IFID_ = 1;
206         flush_IDEX_ = 1;
207         rst_pc_ = 1;
208         ns = S2;
209     end
210     S2:
211     begin
212         unique case(opcode_)
213         `opcode_I : begin
214             write_regf_en_ = 1;
215             case(func3_)
216             0: op_ = `ALUOP_ADD;
217             1: op_ = `ALUOP_SLL;
218             2: op_ = `ALUOP_LT;
219             3: op_ = `ALUOP_LTU;
220             4: op_ = `ALUOP_XOR;
221             5: begin
222                 if(func7_ == 7'h0) op_ = `ALUOP_SRL;else op_ = `ALUOP_SRA;
223             end
224             6: op_ = `ALUOP_OR;
225             7: op_ = `ALUOP_AND;
226             endcase
227         end
228         `opcode_R : begin

```

```

229         if(func7_ != `F7_M) begin
230             write_regf_en_ = 1;
231             sel_alu_b_ = 1;
232             case(func3_)
233             0: begin
234                 if(func7_ == 7'h0) op_ = `ALUOP_ADD;else op_ = `ALUOP_SUB;
235             end
236             1: op_ = `ALUOP_SLL;
237             2: op_ = `ALUOP_LT;
238             3: op_ = `ALUOP_LTU;
239             4: op_ = `ALUOP_XOR;
240             5: begin
241                 if(func7_ == 7'h0) op_ = `ALUOP_SRL;else op_ = `ALUOP_SRA;
242             end
243             6: op_ = `ALUOP_OR;
244             7: op_ = `ALUOP_AND;
245             endcase
246         end
247     else
248     begin
249         if((func3_ == `F_MUL)&(func7_ == `F7_M)) begin
250             write_regf_en_ = 1;
251             sel_rd_value_ = 2;
252         end
253         else if((func3_ == `F_MULHU)&(func7_ == `F7_M)) begin
254             write_regf_en_ = 1;
255             sel_rd_value_ = 2;
256         end
257         else if((func3_ == `F_MULHSU)&(func7_ == `F7_M)) begin
258             write_regf_en_ = 1;
259             sel_rd_value_ = 2;
260         end
261         else if((func3_ == `F_MULH)&(func7_ == `F7_M)) begin
262             write_regf_en_ = 1;
263             sel_rd_value_ = 2;
264         end
265         else if((func3_ == `F_DIV)&(func7_ == `F7_M)) begin
266             write_regf_en_ = 1;

```

```

267         sel_rd_value_ = 3;
268     end
269     else if((funct3_ == `F_DIVU)&(funct7_ == `F7_M)) begin
270         write_regf_en_ = 1;
271         sel_rd_value_ = 3;
272     end
273     else if((funct3_ == `F_REM)&(funct7_ == `F7_M)) begin
274         write_regf_en_ = 1;
275         sel_rd_value_ = 3;
276     end
277     else if((funct3_ == `F_REMU)&(funct7_ == `F7_M)) begin
278         write_regf_en_ = 1;
279         sel_rd_value_ = 3;
280     end
281 end
282 end
283 `opcode_B : begin
284     case(funct3_)
285         F_BEQ : begin
286             if(BEQ_FLAG) begin
287                 sel_pc_ = 1;
288                 flush_IFID_ = 1;
289                 flush_IDEX_ = 1;
290             end
291         end
292         F_BNE : begin
293             if(BNE_FLAG) begin
294                 sel_pc_ = 1;
295                 flush_IFID_ = 1;
296                 flush_IDEX_ = 1;
297             end
298         end
299         F_BLT : begin
300             if(BLT_FLAG) begin
301                 sel_pc_ = 1;
302                 flush_IFID_ = 1;
303                 flush_IDEX_ = 1;
304             end
305         end
306         F_BGE : begin
307             if(BGE_FLAG) begin
308                 sel_pc_ = 1;
309                 flush_IFID_ = 1;
310                 flush_IDEX_ = 1;
311             end
312         end
313         F_BLTU : begin
314             if(BLTU_FLAG) begin
315                 sel_pc_ = 1;
316                 flush_IFID_ = 1;
317                 flush_IDEX_ = 1;
318             end
319         end
320         F_BGEU : begin
321             if(BGEU_FLAG) begin
322                 sel_pc_ = 1;
323                 flush_IFID_ = 1;
324                 flush_IDEX_ = 1;
325             end
326         end
327     endcase
328 end
329 `opcode_JAL : begin
330     sel_pc_ = 1;
331     flush_IFID_ = 1;
332     flush_IDEX_ = 1;
333     sel_alu_a_ = 1;
334     sel_alu_b_ = 2;
335     sel_jump_ = 1;
336     op_ = `ALUOP_ADD;
337     write_regf_en_ = 1;
338 end
339 `opcode_JALR : begin
340     sel_pc_ = 1;

```

```

343         flush_IFID_ = 1;
344         flush_IDEX_ = 1;
345         sel_alu_a_ = 1;
346         sel_alu_b_ = 2;
347         sel_jump_ = 0;
348         op_ = 'ALUOP_ADD;
349         write_regf_en_ = 1;
350
351     end
352     `opcode_LUI : begin
353         sel_alu_b_ = 0;
354         op_ = 'ALUOP_B;
355         write_regf_en_ = 1;
356
357     end
358     `opcode_AUIPC : begin
359         sel_alu_a_ = 1;
360         sel_alu_b_ = 0;
361         op_ = 'ALUOP_ADD;
362         write_regf_en_ = 1;
363
364     end
365     `opcode_L : begin
366         if(func3_ == 'F_LB) begin
367             op_ = 'ALUOP_ADD;
368             sel_rd_value_ = 1;
369             write_regf_en_ = 1;
370         end
371         else if(func3_ == 'F_LH) begin
372             op_ = 'ALUOP_ADD;
373             sel_rd_value_ = 1;
374             write_regf_en_ = 1;
375         end
376         else if(func3_ == 'F_LW) begin
377             op_ = 'ALUOP_ADD;
378             sel_rd_value_ = 1;
379             write_regf_en_ = 1;
380         end
381
382         else if(func3_ == 'F_LBU) begin
383             op_ = 'ALUOP_ADD;
384             sel_rd_value_ = 1;
385             write_regf_en_ = 1;
386         end
387         else if(func3_ == 'F_LHU) begin
388             op_ = 'ALUOP_ADD;
389             sel_rd_value_ = 1;
390             write_regf_en_ = 1;
391         end
392     end
393     `opcode_S : begin
394         if(func3_ == 'F_SB) begin
395             op_ = 'ALUOP_ADD;
396             write_ram_ = 1;
397         end
398         else if(func3_ == 'F_SH) begin
399             op_ = 'ALUOP_ADD;
400             write_ram_ = 1;
401         end
402         else if(func3_ == 'F_SW) begin
403             op_ = 'ALUOP_ADD;
404             write_ram_ = 1;
405         end
406     end
407     default;
408 endcase
409 endcase
410 end
411
412 //reg_file
413 logic write_regf_en_r;
414 logic [4:0] addr_rd_r;
415 logic [31:0] rd_value_;
416
417
418 logic [31:0] regs[0:31];

```

```

419     logic addr_rd_not_0;
420
421     integer i;
422
423     assign addr_rd_not_0 = !addr_rd_r;
424
425     assign rs1_value = regs[addr_rs1_];
426     assign rs2_value = regs[addr_rs2_];
427     //forwarding
428     assign sel_rs1_value_ = write_regf_en_r & (addr_rd_r == addr_rs1_);
429     assign sel_rs2_value_ = write_regf_en_r & (addr_rd_r == addr_rs2_);
430     assign rs1_value_ = sel_rs1_value_ ? rd_value_:rs1_value;
431     assign rs2_value_ = sel_rs2_value_ ? rd_value_:rs2_value;
432
433     always_ff@(posedge clk)
434     begin
435         if(rst) begin
436             for(i = 0; i < 32; i = i+1) begin:rst_keywords
437                 regs[i] <= 0;
438             end
439         end
440         else begin
441             // write
442             if (write_regf_en_r && addr_rd_not_0)
443                 regs[addr_rd_r] <= #1 rd_value_;
444         end
445     end
446     //
447     assign regs_31 = regs[31];
448     //
449     // ID/EX
450     logic [2:0] funct3_r;
451     logic [31:0] imm_r;
452     logic [31:0] rs1_value_r;
453     logic [31:0] rs2_value_r;
454     logic [3:0] op_r;
455     logic [1:0] sel_alu_b_r;
456     logic sel_alu_a_r;

```

```

457     logic sel_jump_r;
458     logic write_ram_r;
459     logic [1:0] sel_rd_value_r;
460     always_ff@(posedge clk)
461     begin
462         if(rst|flush_IDEX_r)
463         begin
464             addr_rd_r <= 5'b0;
465             imm_r <= 32'b0;
466             rs1_value_r <= 32'b0;
467             rs2_value_r <= 32'b0;
468             write_regf_en_r <= 0;
469             flush_IFID_r <= 0;
470             flush_IDEX_r <= 0;
471             pc_rr <= 0;
472             sel_pc_r <= 0;
473             sel_jump_r <= 0;
474             sel_alu_a_r <= 0;
475             sel_alu_b_r <= 0;
476             op_r <= 0;
477             write_ram_r <= 0;
478             sel_rd_value_r <= 0;
479             funct3_r <= 0;
480         end
481         else
482         begin
483             addr_rd_r <= addr_rd_;
484             imm_r <= imm_;
485             rs1_value_r <= rs1_value_;
486             rs2_value_r <= rs2_value_;
487             write_regf_en_r <= write_regf_en_;
488             sel_alu_b_r <= sel_alu_b_;
489             sel_alu_a_r <= sel_alu_a_;
490             op_r <= op_;
491             pc_rr <= pc_r;
492             sel_pc_r <= sel_pc_;
493             flush_IFID_r <= flush_IFID_;
494             flush_IDEX_r <= flush_IDEX_;

```

```

495         sel_jump_r    <= sel_jump_;
496         write_ram_r    <= write_ram_;
497         sel_rd_value_r <= sel_rd_value_;
498         funct3_r       <= funct3_;
499     end
500 end
501 logic [31:0] base_addr_;
502 logic [31:0] jump_addr_;
503 logic [31:0] j_addr_;
504 assign base_addr_ = sel_jump_r ? pc_rr : rs1_value_r;
505 assign j_addr_ = base_addr_ + imm_r;
506 assign jump_addr_ = {j_addr_[31:1], (j_addr_[0] & sel_jump_r)};
507
508 always_comb
509 begin
510     if(sel_pc_r)
511         pc_next_ = jump_addr_;
512     else
513         pc_next_ = pc + 4;
514     end
515 //ALU
516
517 logic [31:0] alu_a_;
518 logic [31:0] alu_b_;
519 logic [31:0] alu_out_;
520 logic [31:0] div_out;
521 logic [31:0] mul_out;
522 assign alu_a_ = sel_alu_a_r ? pc_rr : rs1_value_r;
523 always_comb
524 begin
525     if(sel_alu_b_r == 0)
526         alu_b_ = imm_r;
527     else if(sel_alu_b_r == 1)
528         alu_b_ = rs2_value_r;
529     else if(sel_alu_b_r == 2)
530         alu_b_ = 4;
531     else
532         alu_b_ = 4;

```

```

533     end
534
535     alu alu1 (
536         .op      (op_r      ),
537         .alu_a    (alu_a_    ),
538         .alu_b    (alu_b_    ),
539         .alu_out  (alu_out_)
540     );
541
542     mul mul1 (
543         .funct3   (funct3_r   ),
544         .rs1_value (rs1_value_r ),
545         .rs2_value (rs2_value_r ),
546         .mul_out  (mul_out   )
547     );
548
549     div div1 (
550         .funct3   (funct3_r   ),
551         .rs1_value (rs1_value_r ),
552         .rs2_value (rs2_value_r ),
553         .div_out  (div_out   )
554     );
555 //load and store unit
556 logic [31:0] read_data;
557 LSU LSU1 (
558     .clk      (clk      ),
559     .write_ram (write_ram_r ),
560     .funct3    (funct3_r   ),
561     .write_data (rs2_value_r ),
562     .ram_addr  (alu_out_   ),
563     .read_data (read_data  )
564 );
565 always_comb
566 begin
567     if(sel_rd_value_r == 0)
568         rd_value_ = alu_out_;
569     else if(sel_rd_value_r == 1)
570         rd_value_ = read_data;
571
572     else if(sel_rd_value_r == 2)
573         rd_value_ = mul_out;
574     else
575         rd_value_ = div_out;
576 end
577 endmodule
578

```

program_rom

```

1 module Program_Rom(
2     output logic [31:0] Rom_data,
3     input [31:0] Rom_addr
4 );
5
6 always_comb begin
7     case (Rom_addr)
8         32'h00000000 : Rom_data = 32'h341CE137; // lui x2 213454
9         32'h00000004 : Rom_data = 32'hF0C10113; // addi x2 x2 -244
10        32'h00000008 : Rom_data = 32'h00202023; // sw x2 0(x0)
11        32'h0000000C : Rom_data = 32'hBD006137; // lui x2 774150
12        32'h00000010 : Rom_data = 32'h36510113; // addi x2 x2 869
13        32'h00000014 : Rom_data = 32'h00202223; // sw x2 4(x0)
14        32'h00000018 : Rom_data = 32'h00F00113; // addi x2 x0 15
15        32'h0000001C : Rom_data = 32'h00200423; // sb x2 8(x0)
16        32'h00000020 : Rom_data = 32'h00900193; // addi x3 x0 9
17        32'h00000024 : Rom_data = 32'h6E056137; // lui x2 450646
18        32'h00000028 : Rom_data = 32'hFF510113; // addi x2 x2 -11
19        32'h0000002C : Rom_data = 32'h0021A023; // sw x2 0(x3)
20        32'h00000030 : Rom_data = 32'h34435137; // lui x2 214069
21        32'h00000034 : Rom_data = 32'hDF710113; // addi x2 x2 -521
22        32'h00000038 : Rom_data = 32'h0021A223; // sw x2 4(x3)
23        32'h0000003C : Rom_data = 32'h00C00113; // addi x2 x0 12
24        32'h00000040 : Rom_data = 32'h00218423; // sb x2 8(x3)
25        32'h00000044 : Rom_data = 32'h01200193; // addi x3 x0 18
26        32'h00000048 : Rom_data = 32'h00300213; // addi x4 x0 3
27        32'h0000004C : Rom_data = 32'h00000313; // addi x6 x0 0
28        32'h00000050 : Rom_data = 32'h00300293; // addi x5 x0 3
29        32'h00000054 : Rom_data = 32'h00900393; // addi x7 x0 9
30        32'h00000058 : Rom_data = 32'hFFF28293; // addi x5 x5 -1
31        32'h0000005C : Rom_data = 32'h00030583; // lb x11 0(x6)
32        32'h00000060 : Rom_data = 32'h00038603; // lb x12 0(x7)
33        32'h00000064 : Rom_data = 32'h02C58433; // mul x8 x11 x12
34        32'h00000068 : Rom_data = 32'h00130583; // lb x11 1(x6)
35        32'h0000006C : Rom_data = 32'h00338603; // lb x12 3(x7)
36        32'h00000070 : Rom_data = 32'h02C58483; // mul x9 x11 x12
37        32'h00000074 : Rom_data = 32'h00230583; // lb x11 2(x6)
38        32'h00000078 : Rom_data = 32'h00638603; // lb x12 6(x7)
39
40        32'h0000007C : Rom_data = 32'h02C58533; // mul x10 x11 x12
41        32'h00000080 : Rom_data = 32'h00A40533; // add x10 x8 x10
42        32'h00000084 : Rom_data = 32'h00A48533; // add x10 x9 x10
43        32'h00000088 : Rom_data = 32'h00138393; // addi x7 x7 1
44        32'h0000008C : Rom_data = 32'h00A19023; // sh x10 0(x3)
45        32'h00000090 : Rom_data = 32'h00218193; // addi x3 x3 2
46        32'h00000094 : Rom_data = 32'hFC0292E3; // bne x5 x0 -60
47        32'h00000098 : Rom_data = 32'h00330313; // addi x6 x6 3
48        32'h0000009C : Rom_data = 32'hFFF20213; // addi x4 x4 -1
49        32'h000000A0 : Rom_data = 32'hFA0218E3; // bne x4 x0 -80
50        32'h000000A4 : Rom_data = 32'h01200193; // addi x3 x0 18
51        32'h000000A8 : Rom_data = 32'h00900213; // addi x4 x0 9
52        32'h000000AC : Rom_data = 32'h00019F83; // lh x31 0(x3)
53        32'h000000B0 : Rom_data = 32'h00218193; // addi x3 x3 2
54        32'h000000B4 : Rom_data = 32'hFFF20213; // addi x4 x4 -1
55        32'h000000B8 : Rom_data = 32'hFE021AE3; // bne x4 x0 -12
56        default : Rom_data = 32'h00000013; // NOP
57    endcase
58 end
59 endmodule
60

```

testbench

```

1 module testbench;
2
3
4
5     logic clk,rst;
6
7
8     risc_v al(
9         .clk(clk),
10        .rst(rst)
11    );
12
13
14    always #10 clk = ~clk;
15    initial begin
16        clk = 0; rst = 1;
17        #40 rst = 0;
18        #200000 $stop;
19    end
20 endmodule

```

```

ASM Matrix_mul.s
1  init:
2      li x2, 0x341CDF0C
3      sw x2, 0(x0)
4      li x2, 0xBD006365
5      sw x2, 4(x0)
6      li x2, 0xF
7      sb x2, 8(x0)
8      li x3, 0x9
9      li x2, 0x6E055FF5
10     sw x2, 0(x3)
11     li x2, 0x34434DF7
12     sw x2, 4(x3)
13     li x2, 0xC
14     sb x2, 8(x3)
15
16     // 準備迴圈參數
17     addi x3, x0, 0x12    // x3 = 0x12 (結果存放起始位址)
18     addi x4, x0, 3      // x4 = 3 (矩陣的列數 rows)
19     addi x6, x0, 0      // x6 = 0 (矩陣目前列的offset)
20
21     loop1: // 外部迴圈 (遍歷矩陣 A 的每一列)
22         addi x5, x0, 3    // x5 = 3 (矩陣的行數 columns)
23         addi x7, x0, 9    // x7 = 9 (矩陣目前行的起始位址, 初始為 0x9)
24
25     loop2:
26         addi x5, x5, -1
27
28         //計算第 1 個乘積項 (A[row][0] * B[0][col])

```

```

28         //計算第 1 個乘積項 (A[row][0] * B[0][col])
29         lb x11, 0(x6)
30         lb x12, 0(x7)
31         mul x8, x11, x12    // 乘積存入 x8
32
33         //計算第 2 個乘積項 (A[row][1] * B[1][col])
34         lb x11, 1(x6)
35         lb x12, 3(x7)
36         mul x9, x11, x12    // 乘積存入 x9
37
38         //計算第 3 個乘積項 (A[row][2] * B[2][col])
39         lb x11, 2(x6)
40         lb x12, 6(x7)
41         mul x10, x11, x12    // 乘積存入 x10
42
43         // --- 加總並儲存 ---
44         add x10, x8, x10    // 累加
45         add x10, x9, x10    // 累加總和至 x10
46
47         addi x7, x7, 1      // 移動到下一行
48         sh x10, 0(x3)      // 將結果存入矩陣目前位置
49         addi x3, x3, 2
50
51         bne x5, x0, loop2
52
53         // 外部迴圈結尾處理

```

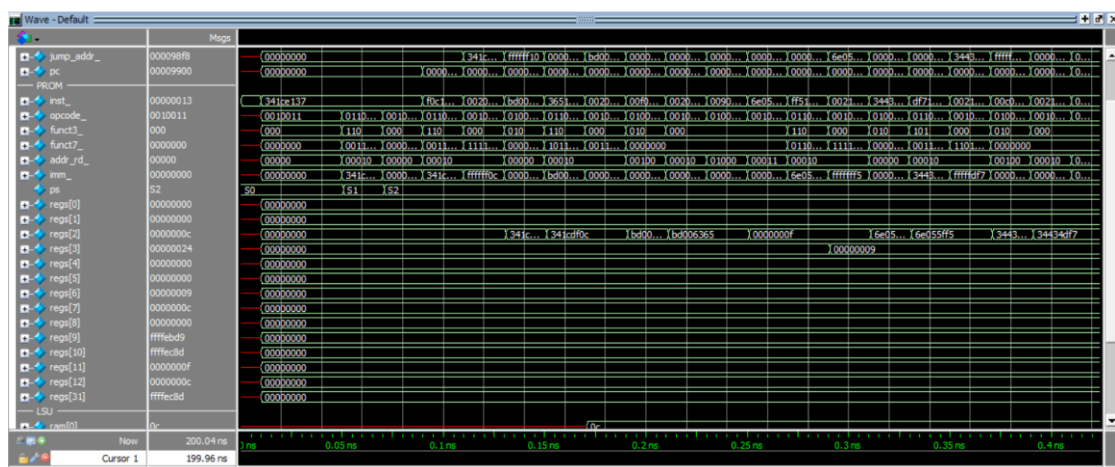
```

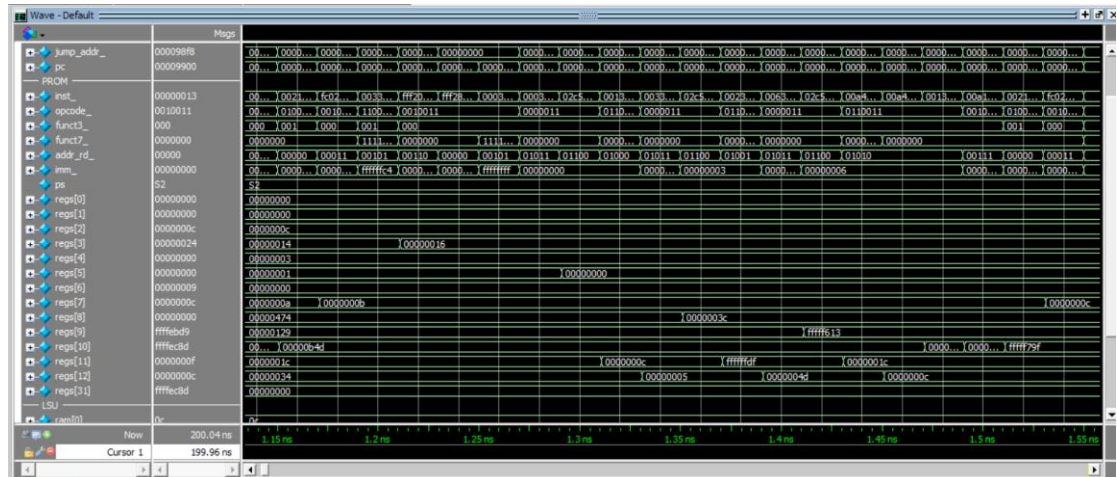
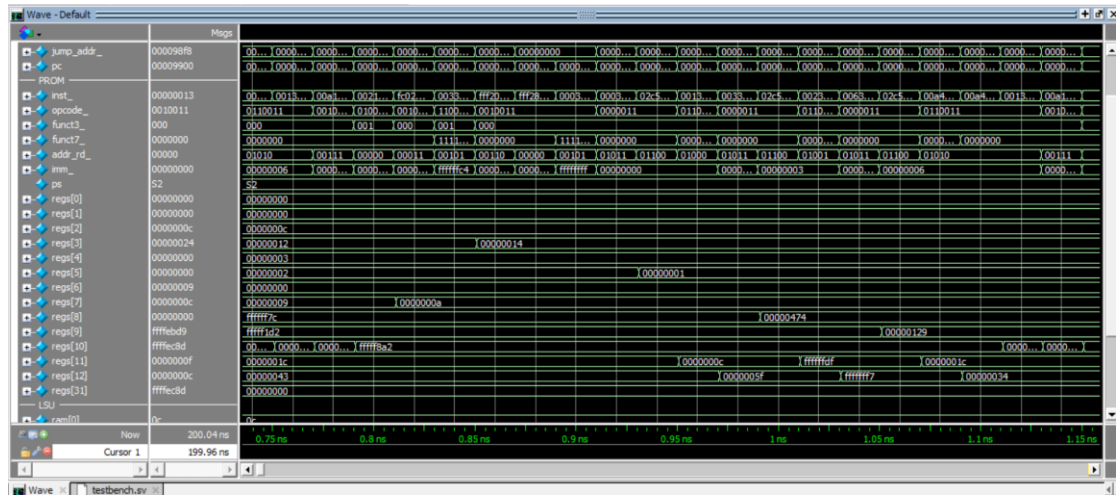
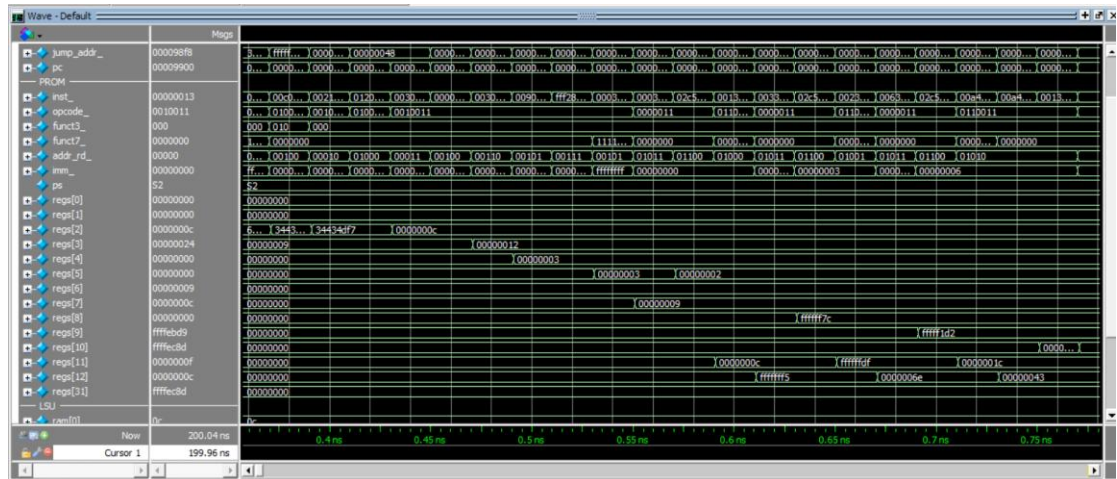
54      addi x6, x6, 3
55      addi x4, x4, -1
56      bne x4, x0, loop1
57
58      li x3, 0x12
59      li x4, 9
60
61  done:
62      lh x31, 0(x3)
63      addi x3, x3, 2
64      addi x4, x4, -1
65      bne x4, x0, done

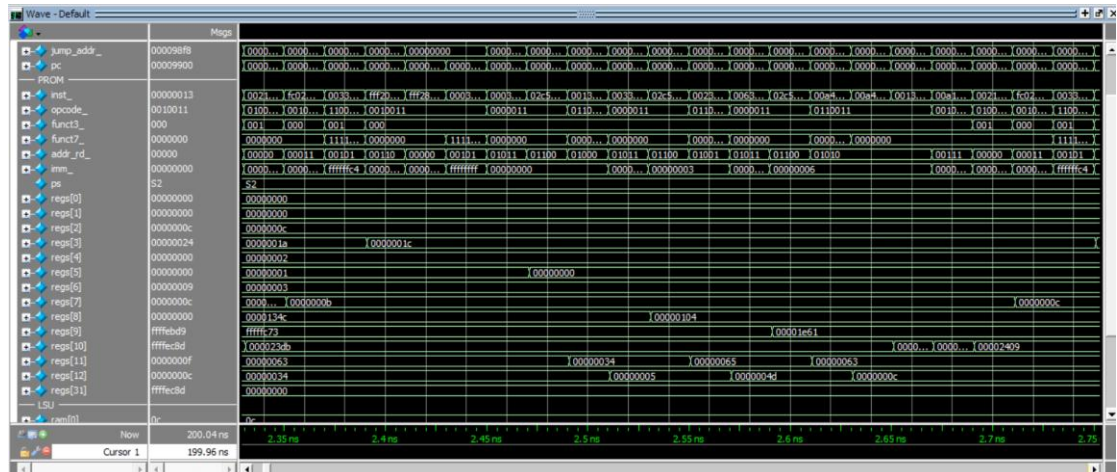
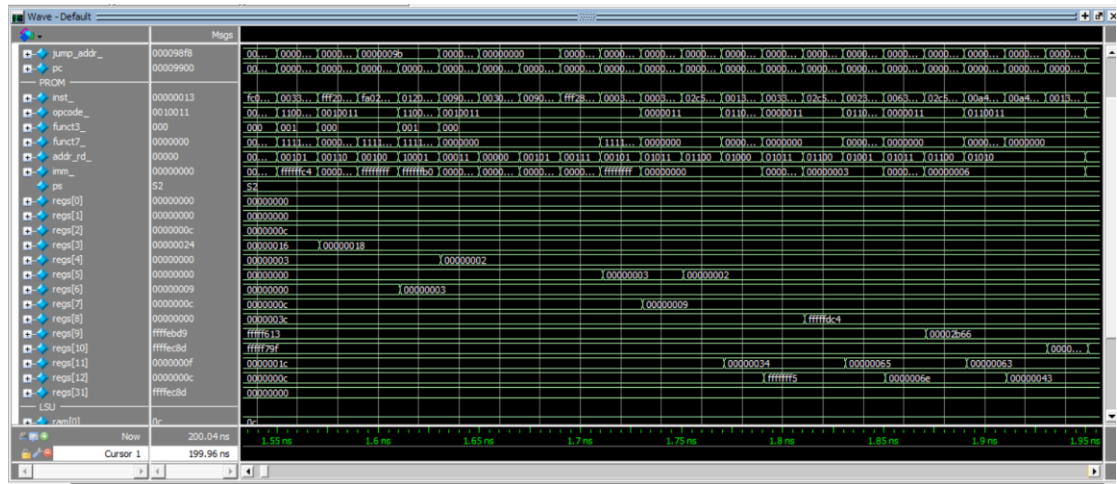
```

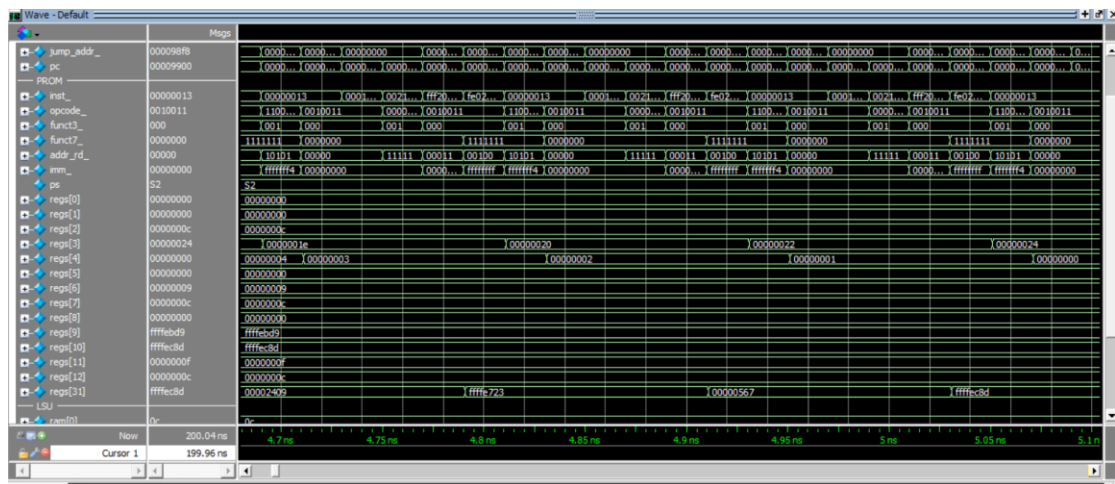
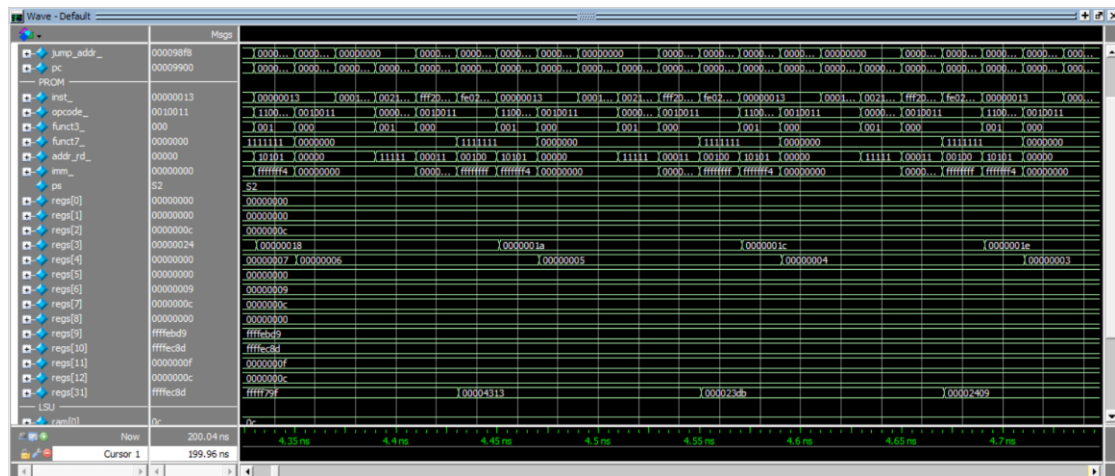
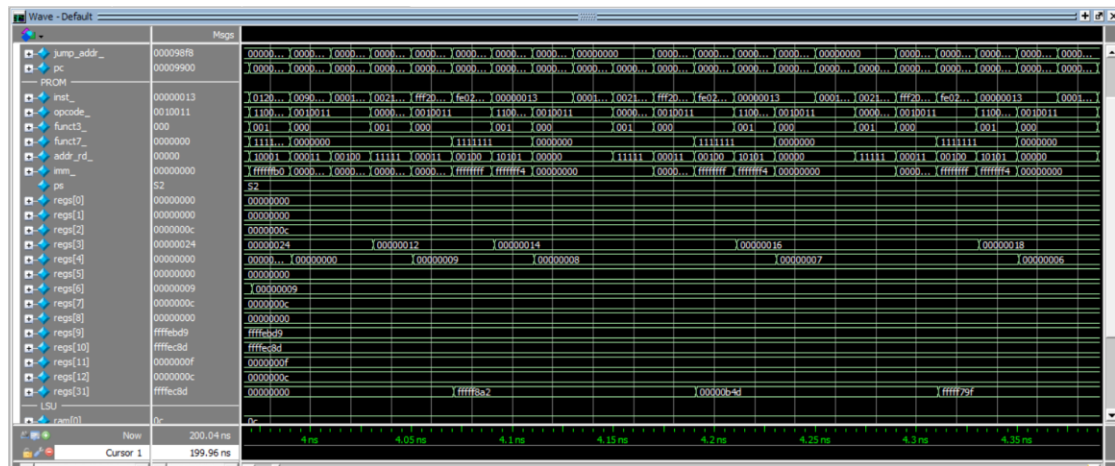
- [x3] 1. 初始化時：指向矩陣 B 的寫入起始位址 (0x9)。
2. 運算時：指向結果矩陣 C 的存放位址
3. 結束時：用於讀取 C 的結果以供觀察。
- [x4] 控制矩陣 A 的列 (Row) 數。
用途：初始化為 3，每處理完矩陣 A 的一列後遞減，直到為 0。
- [x5] 控制矩陣 B 的行 (Column) 數。
- [x6] 指向矩陣 A 目前「列」的起始位置。
- [x7] 指向矩陣 B 目前「行」的起始位置。
- [x8] 存放 (A[row][0] * B[0][col]) 的乘積。
- [x9] 存放 (A[row][1] * B[1][col]) 的乘積。
- [x10] 先存放 (A[row][2] * B[2][col])，接著將 x8, x9 加進來，成為最終的內積結果。
- [x11] 資料載入暫存。
- [x12] 資料載入暫存。
- [x31] 將計算結果依序載入此暫存器。

● 模擬結果與結果說明：









- [x3] 1. 初始化時：指向矩陣 B 的寫入起始位址 (0x9)。
2. 運算時：指向結果矩陣 C 的存放位址
3. 結束時：用於讀取 C 的結果以供觀察。
- [x4] 控制矩陣 A 的列 (Row) 數。
用途：初始化為 3，每處理完矩陣 A 的一列後遞減，直到為 0。
- [x5] 控制矩陣 B 的行 (Column) 數。
- [x6] 指向矩陣 A 目前「列」的起始位置。

- [x7] 指向矩陣 B 目前「行」的起始位置。
- [x8] 存放 $(A[\text{row}][0] * B[0][\text{col}])$ 的乘積。
- [x9] 存放 $(A[\text{row}][1] * B[1][\text{col}])$ 的乘積。
- [x10] 先存放 $(A[\text{row}][2] * B[2][\text{col}])$ ，接著將 x8, x9 加進來，成為最終的內積結果。
- [x11] 資料載入暫存。
- [x12] 資料載入暫存。
- [x31] 將計算結果依序載入此暫存器。

● 結論與心得：

這次 LI 可以正常使用了，特別順，不用再分開寫了，還要慢慢加好幾次，才可以存入暫存器。